

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**

Факультет информационных технологий
Кафедра информатики и информационных технологий
Направление подготовки 09.03.02
«Информационные системы и технологии»

ЛАБОРАТОРНАЯ РАБОТА № 1

Дисциплина: «Функциональное программирование»

Тема: «Основные принципы функционального программирования
в JavaScript. Чистые функции и функции высшего порядка»

Выполнила: студентка группы 241-335

Басимова Анастасия Маратовна

Дата, подпись _____ / _____

Проверил: _____

Дата, подпись _____ / _____

Замечания: _____

Москва, 2026

1 ЦЕЛЬ РАБОТЫ

Освоить основные принципы функционального программирования в языке JavaScript, научиться создавать чистые функции и использовать функции высшего порядка.

2 ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

1. Разработать набор чистых функций для работы с массивами:

- функция, которая принимает массив чисел и возвращает новый массив, содержащий только чётные числа;
- функция, которая принимает массив чисел и возвращает новый массив, содержащий квадраты этих чисел;
- функция, которая принимает массив объектов и возвращает новый массив, содержащий только объекты с определённым свойством;
- функция, которая принимает массив чисел и возвращает их сумму.

2. Создать функцию высшего порядка, которая принимает функцию и массив в качестве аргументов и применяет функцию к каждому элементу массива, возвращая новый массив с результатами.

3. Используя разработанные функции, выполнить следующие математические операции:

- найти сумму квадратов всех чётных чисел в заданном массиве;
- найти среднее арифметическое всех чисел, больших заданного значения, в заданном массиве объектов.

Требования к реализации:

- все функции должны быть чистыми, то есть не иметь побочных эффектов и всегда возвращать одинаковый результат при одинаковых аргументах;

- использовать стрелочные функции и другие современные возможности языка JavaScript;
- код должен быть хорошо оформлен и легко читаем.

3 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Функциональное программирование – парадигма, в которой вычисления представляются как применение функций к аргументам, а изменяемое состояние и побочные эффекты сводятся к минимуму.

Чистой (pure) называется функция, которая удовлетворяет двум условиям: при одних и тех же аргументах она всегда возвращает один и тот же результат (детерминированность) и не производит побочных эффектов, то есть не изменяет внешние переменные и переданные ей аргументы, не выполняет ввод-вывод. Благодаря этому чистые функции легко тестировать и безопасно комбинировать между собой.

Функцией высшего порядка называется функция, которая принимает другие функции в качестве аргументов и (или) возвращает функцию как результат. Встроенными функциями высшего порядка в JavaScript являются методы массива `map`, `filter` и `reduce`.

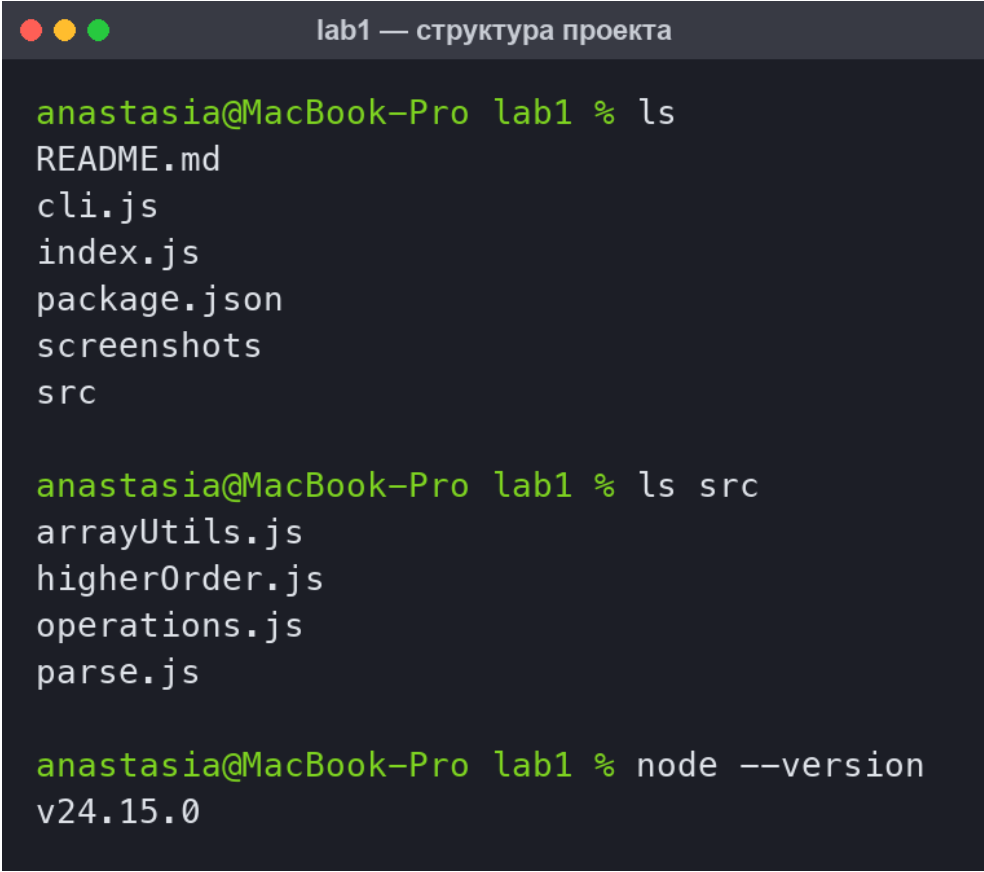
Для реализации использованы современные возможности стандарта ECMAScript: стрелочные функции, модули ES (`import/export`), оператор расширения (`spread`), оператор возведения в степень, шаблонные строки.

4 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

4.1 Структура проекта

Программа разработана на языке JavaScript и выполняется средствами платформы Node.js. Код разделён на модули: модуль чистых функций для работы с массивами, модуль функции высшего порядка, модуль математических операций и модуль разбора введённых данных. Предусмотрено

два режима запуска: файл `index.js` демонстрирует работу на предустановленных данных, а файл `cli.js` позволяет пользователю ввести свои данные. Структура проекта приведена на рисунке 1.



```
lab1 — структура проекта

anastasia@MacBook-Pro lab1 % ls
README.md
cli.js
index.js
package.json
screenshots
src

anastasia@MacBook-Pro lab1 % ls src
arrayUtils.js
higherOrder.js
operations.js
parse.js

anastasia@MacBook-Pro lab1 % node --version
v24.15.0
```

Рисунок 1 – Структура проекта и версия среды выполнения

4.2 Чистые функции для работы с массивами

В модуле `src/arrayUtils.js` реализованы четыре чистые функции. Каждая из них возвращает новый массив (или число) и не изменяет переданные аргументы: методы `filter`, `map` и `reduce` не модифицируют исходный массив, а создают новое значение. Листинг модуля приведён ниже.

Листинг 1 – Модуль `src/arrayUtils.js`

```
/**
 * Набор чистых функций для работы с массивами.
 * Все функции не изменяют входные данные и не имеют побочных эффектов.
 */
```

```

/**
 * Возвращает новый массив, содержащий только чётные числа.
 * @param {number[]} numbers - исходный массив чисел
 * @returns {number[]} новый массив чётных чисел
 */
export const filterEven = (numbers) => numbers.filter((n) => n % 2 === 0);

/**
 * Возвращает новый массив квадратов исходных чисел.
 * @param {number[]} numbers - исходный массив чисел
 * @returns {number[]} новый массив квадратов
 */
export const squareAll = (numbers) => numbers.map((n) => n ** 2);

/**
 * Возвращает новый массив объектов с заданным свойством.
 * @param {Object[]} items - исходный массив объектов
 * @param {string} property - имя искомого свойства
 * @returns {Object[]} новый массив отфильтрованных объектов
 */
export const filterByProperty = (items, property) =>
  items.filter((item) =>
    Object.prototype.hasOwnProperty.call(item, property),
  );

/**
 * Возвращает сумму элементов массива чисел.
 * @param {number[]} numbers - исходный массив чисел
 * @returns {number} сумма элементов
 */
export const sum = (numbers) => numbers.reduce((acc, n) => acc + n, 0);

```

4.3 Функция высшего порядка

Функция высшего порядка `applyToEach` принимает функцию `fn` и массив `items`, применяет `fn` к каждому элементу массива и возвращает новый массив с результатами. Реализация выполнена через `reduce` с использованием оператора расширения, что исключает мутацию аккумулятора.

Листинг 2 – Модуль `src/higherOrder.js`

```

/**
 * Функции высшего порядка.
 */

/**
 * Применяет переданную функцию к каждому элементу массива
 * и возвращает новый массив с результатами (аналог Array.prototype.map).
 * @param {Function} fn - применяемая функция (value, index, array)
 * @param {*}[] items - исходный массив
 * @returns {*}[] новый массив результатов

```

```

*/
export const applyToEach = (fn, items) =>
  items.reduce((acc, item, index) => [...acc, fn(item, index, items)], []);

```

4.4 Математические операции

Требуемые математические операции построены путём композиции ранее разработанных функций. Сумма квадратов чётных чисел вычисляется как последовательное применение `filterEven`, `applyToEach` и `sum`. Среднее арифметическое значений, превышающих заданный порог, вычисляется как отношение суммы отобранных значений к их количеству; при отсутствии подходящих элементов возвращается ноль, что исключает деление на ноль.

Листинг 3 – Модуль `src/operations.js`

```

/**
 * Математические операции, построенные из ранее описанных чистых функций.
 */

import { filterEven, filterByProperty, sum } from './arrayUtils.js';
import { applyToEach } from './higherOrder.js';

/**
 * Сумма квадратов всех чётных чисел массива.
 * @param {number[]} numbers - исходный массив чисел
 * @returns {number} сумма квадратов чётных чисел
 */
export const sumOfEvenSquares = (numbers) =>
  sum(applyToEach((n) => n ** 2, filterEven(numbers)));

/**
 * Среднее арифметическое значений свойства, превышающих заданный порог.
 * @param {Object[]} items - массив объектов
 * @param {string} property - имя числового свойства
 * @param {number} threshold - пороговое значение
 * @returns {number} среднее арифметическое (0, если элементов нет)
 */
export const averageAboveThreshold = (items, property, threshold) => {
  const values = applyToEach(
    (item) => item[property],
    filterByProperty(items, property),
  ).filter((value) => value > threshold);

  return values.length === 0 ? 0 : sum(values) / values.length;
};

```

4.5 Точка входа программы

В файле `index.js` заданы исходные данные – массив целых чисел от 1 до 10 и массив объектов, описывающих сотрудников, причём у части объектов свойство `salary` отсутствует. Демонстрация разбита на четыре раздела, номера которых можно передать программе аргументами командной строки.

Листинг 4 – Файл `index.js`

```
/**
 * Лабораторная работа №1
 * Основные принципы функционального программирования в JavaScript.
 *
 * Точка входа: демонстрация работы чистых функций, функции высшего порядка
 * и построенных на их основе математических операций.
 */

import {
  filterEven,
  squareAll,
  filterByProperty,
  sum,
} from './src/arrayUtils.js';
import { applyToEach } from './src/higherOrder.js';
import {
  sumOfEvenSquares,
  averageAboveThreshold,
} from './src/operations.js';
import { inspect } from 'node:util';

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

const employees = [
  { name: 'Иванов', salary: 45000, department: 'IT' },
  { name: 'Петров', salary: 72000, department: 'IT' },
  { name: 'Сидорова', salary: 58000 },
  { name: 'Кузнецов', department: 'HR' },
  { name: 'Смирнова', salary: 91000, department: 'Finance' },
];

const format = (value) =>
  Array.isArray(value)
    ? `[ ${value.map((item) => inspect(item)).join(', ')} ]`
    : inspect(value);

const show = (label, value) => console.log(label.padEnd(32), format(value));

/** Демонстрационные разделы: номер раздела -> функция вывода. */
const sections = {
  1: () => {
    console.log('=== 1. Чистые функции для работы с массивами ===');
```

```

    show('Исходный массив:', numbers);
    show('filterEven:', filterEven(numbers));
    show('squareAll:', squareAll(numbers));
    show('sum:', sum(numbers));
    console.log("filterByProperty(employees, 'salary'):");
    console.log(filterByProperty(employees, 'salary'));
  },
  2: () => {
    console.log('=== 2. Функция высшего порядка applyToEach ===');
    show('Удвоение элементов:', applyToEach((n) => n * 2, numbers));
    show('Приведение к строке:', applyToEach((n) => `№${n}`, [1, 2, 3]));
    show(
      'Имена сотрудников:',
      applyToEach((employee) => employee.name, employees),
    );
  },
  3: () => {
    console.log('=== 3. Математические операции ===');
    show('Сумма квадратов чётных чисел:', sumOfEvenSquares(numbers));
    show(
      'Среднее зарплат > 50000:',
      averageAboveThreshold(employees, 'salary', 50000),
    );
  },
  4: () => {
    console.log('=== 4. Проверка чистоты функций ===');
    const original = [1, 2, 3, 4];
    const firstCall = filterEven(original);
    const secondCall = filterEven(original);
    show('Исходный массив не изменён:', original);
    show(
      'Результаты двух вызовов равны:',
      JSON.stringify(firstCall) === JSON.stringify(secondCall),
    );
    show('Возвращён новый массив:', firstCall !== secondCall);
  },
};

const requested = process.argv.slice(2);
const keys = requested.length > 0 ? requested : Object.keys(sections);

keys.forEach((key, index) => {
  const section = sections[key];

  if (section === undefined) {
    console.log(`Раздел «${key}» не найден. Разделы: 1, 2, 3, 4.`);
    return;
  }

  if (index > 0) {
    console.log('');
  }

  section();
});

```

4.6 Интерактивный ввод исходных данных

Чтобы программу можно было проверить на произвольных данных, реализован интерактивный режим. Разбор введённых строк выполняют чистые функции модуля `src/parse.js`: функция `parseNumbers` преобразует строку в массив чисел, отбрасывая нераспознанные элементы, а `parseObjects` — в массив объектов, причём элемент без двоеточия даёт объект без свойства `value`. Это позволяет продемонстрировать работу функции `filterByProperty` на данных, введённых пользователем.

Листинг 5 – Модуль `src/parse.js`

```
/**
 * Чистые функции разбора данных, введённых пользователем.
 * Разбор строки не имеет побочных эффектов: одна и та же
 * строка всегда даёт одинаковый результат.
 */

/**
 * Преобразует строку в массив чисел.
 * Разделители – пробелы, запятые и точки с запятой.
 * @param {string} input - строка, введённая пользователем
 * @returns {number[]} массив чисел
 */
export const parseNumbers = (input) =>
  input
    .split(/[\\s,;]+/)
    .filter((part) => part.length > 0)
    .map(Number)
    .filter((value) => Number.isFinite(value));

/**
 * Преобразует строку в массив объектов.
 * Элемент вида «имя:значение» даёт объект со свойством value,
 * элемент без двоеточия – объект без этого свойства.
 * @param {string} input - строка, введённая пользователем
 * @returns {Object[]} массив объектов
 */
export const parseObjects = (input) =>
  input
    .split(/[\\s,;]+/)
    .filter((part) => part.length > 0)
    .map((part) => part.split(':'))
    .map(([name, value]) =>
      value === undefined || !Number.isFinite(Number(value))
        ? { name }
        : { name, value: Number(value) },
    );
```

Файл `cli.js` запрашивает данные и выводит результаты. Ввод-вывод является побочным эффектом, поэтому он сосредоточен в этом файле, а все вычисления по-прежнему выполняют чистые функции из каталога `src`. Такое разделение позволяет менять способ получения данных, не затрагивая логику обработки.

Листинг 6 – Файл `cli.js`

```
/**
 * Интерактивный режим: пользователь сам вводит исходные данные.
 * Ввод-вывод – единственный побочный эффект программы,
 * все вычисления выполняют чистые функции из каталога src.
 */

import { createInterface } from 'node:readline/promises';
import { inspect } from 'node:util';

import {
  filterEven,
  squareAll,
  filterByProperty,
  sum,
} from './src/arrayUtils.js';
import { applyToEach } from './src/higherOrder.js';
import {
  sumOfEvenSquares,
  averageAboveThreshold,
} from './src/operations.js';
import { parseNumbers, parseObjects } from './src/parse.js';

const format = (value) =>
  Array.isArray(value)
    ? `[ ${value.map((item) => inspect(item)).join(', ')} ]`
    : inspect(value);

const show = (label, value) =>
  console.log(label.padEnd(32), format(value));

const rl = createInterface({
  input: process.stdin,
  output: process.stdout,
});

console.log('=== Обработка массива чисел ===');
const numbers = parseNumbers(
  await rl.question('Введите числа через пробел: '),
);

if (numbers.length === 0) {
  console.log('Числа не распознаны, работа завершена.');
```

```

    process.exit(0);
}

show('Введённый массив:', numbers);
show('Только чётные числа:', filterEven(numbers));
show('Квадраты чисел:', squareAll(numbers));
show('Сумма чисел:', sum(numbers));
show('Сумма квадратов чётных:', sumOfEvenSquares(numbers));
show('Утроение через applyToEach:', applyToEach((n) => n * 3, numbers));

console.log('\n=== Обработка массива объектов ===');
console.log('Формат: имя:значение (элемент без значения — объект');
console.log('без свойства value). Например: Иванов:45000 Кузнецов');

const objects = parseObjects(
    await rl.question('Введите объекты: '),
);

console.log('Введённые объекты:');
console.log(objects);
console.log("Объекты со свойством value:");
console.log(filterByProperty(objects, 'value'));

const threshold = Number(await rl.question('Введите порог: '));

if (Number.isFinite(threshold)) {
    show(
        `Среднее значений > ${threshold}:`,
        averageAboveThreshold(objects, 'value', threshold),
    );
} else {
    console.log('Порог не распознан.');
```

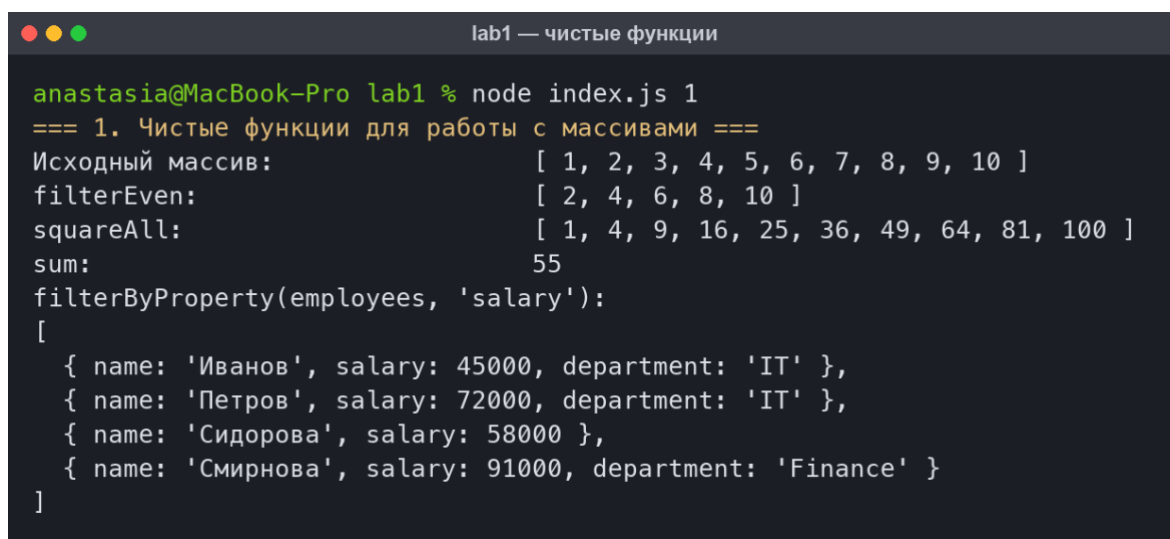
```

rl.close();

```

5 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

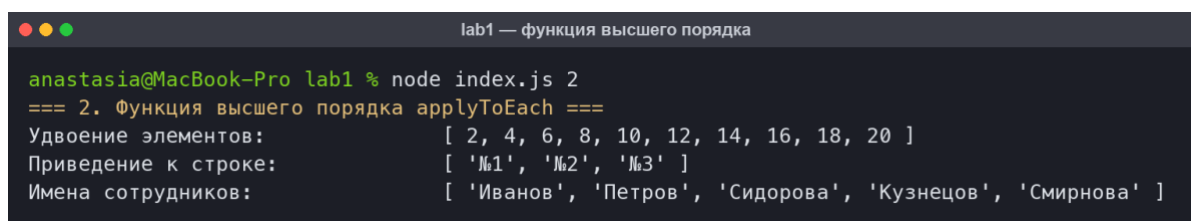
Результат работы чистых функций приведён на рисунке 2. Функция `filterEven` отобрала из массива только чётные числа, функция `squareAll` вернула массив квадратов, функция `sum` вычислила сумму всех элементов (55), а функция `filterByProperty` оставила только те объекты, у которых присутствует свойство `salary` – объект «Кузнецов» без этого свойства в результат не попал.



```
anastasia@MacBook-Pro lab1 % node index.js 1
=== 1. Чистые функции для работы с массивами ===
Исходный массив:      [ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ]
filterEven:           [ 2, 4, 6, 8, 10 ]
squareAll:            [ 1, 4, 9, 16, 25, 36, 49, 64, 81, 100 ]
sum:                  55
filterByProperty(employees, 'salary'):
[
  { name: 'Иванов', salary: 45000, department: 'IT' },
  { name: 'Петров', salary: 72000, department: 'IT' },
  { name: 'Сидорова', salary: 58000 },
  { name: 'Смирнова', salary: 91000, department: 'Finance' }
]
```

Рисунок 2 – Работа чистых функций для работы с массивами

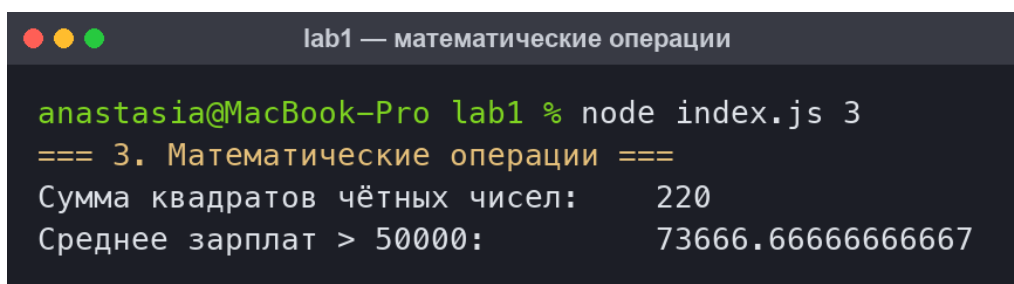
Работа функции высшего порядка `applyToEach` показана на рисунке 3. Одна и та же функция применена с разными аргументами: удвоение чисел, преобразование числа в строку и извлечение свойства `name` из объектов, – что подтверждает универсальность реализации.



```
anastasia@MacBook-Pro lab1 % node index.js 2
=== 2. Функция высшего порядка applyToEach ===
Удвоение элементов:   [ 4, 8, 12, 16, 20, 24, 28, 32, 36, 40 ]
Приведение к строке:  [ '2', '4', '6', '8', '10', '12', '14', '16', '18', '20' ]
Имена сотрудников:    [ 'Иванов', 'Петров', 'Сидорова', 'Кузнецов', 'Смирнова' ]
```

Рисунок 3 – Работа функции высшего порядка `applyToEach`

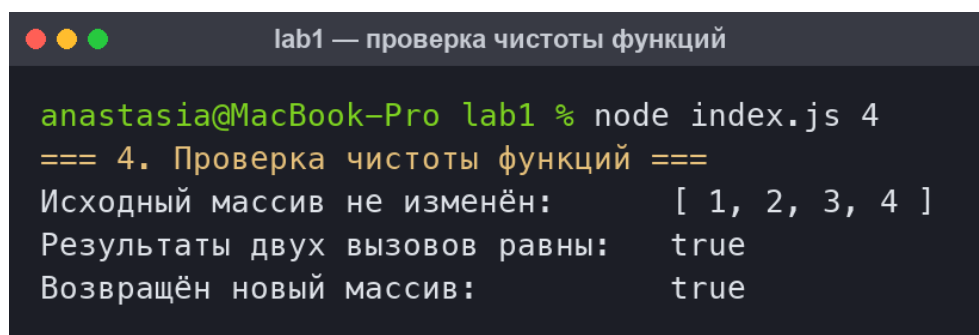
Результаты математических операций приведены на рисунке 4. Сумма квадратов чётных чисел массива от 1 до 10 равна 220, что совпадает с проверочным расчётом: $4 + 16 + 36 + 64 + 100 = 220$. Среднее арифметическое зарплат, превышающих 50 000, составляет 73 666,67, что также соответствует ручной проверке: $(72\ 000 + 58\ 000 + 91\ 000) / 3$.



```
anastasia@MacBook-Pro lab1 % node index.js 3
=== 3. Математические операции ===
Сумма квадратов чётных чисел:      220
Среднее зарплат > 50000:          73666.66666666667
```

Рисунок 4 – Результаты математических операций

Проверка чистоты разработанных функций приведена на рисунке 5. После вызова функции исходный массив остался неизменным, два последовательных вызова с одинаковыми аргументами дали одинаковый результат, при этом каждый вызов возвращает новый объект массива, а не ссылку на исходный.

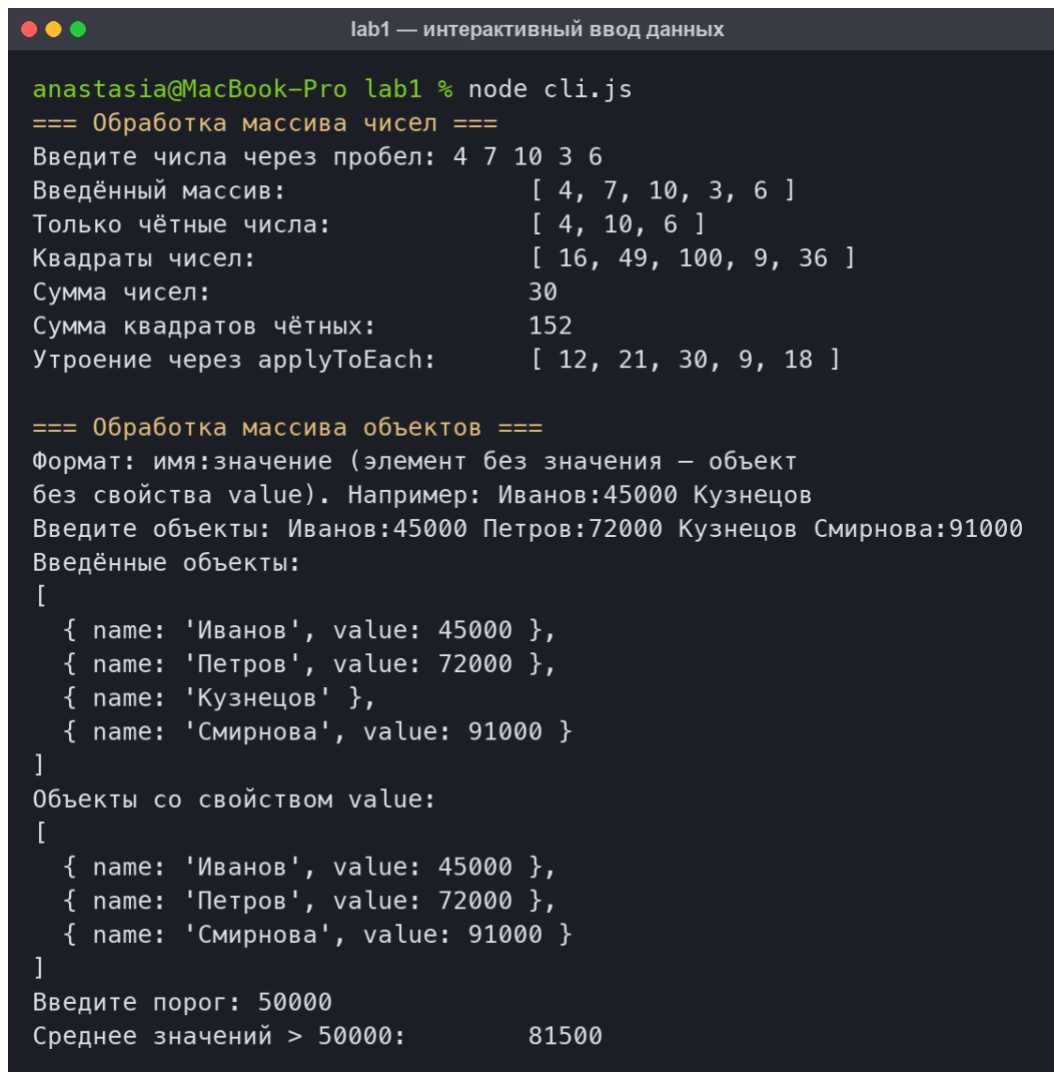


```
anastasia@MacBook-Pro lab1 % node index.js 4
=== 4. Проверка чистоты функций ===
Исходный массив не изменён:      [ 1, 2, 3, 4 ]
Результаты двух вызовов равны:   true
Возвращён новый массив:          true
```

Рисунок 5 – Проверка чистоты функций и неизменности данных

Работа интерактивного режима приведена на рисунке 6. Пользователь ввёл массив чисел 4, 7, 10, 3, 6 и массив объектов, у одного из которых свойство value отсутствует. Программа отобрала чётные числа, вычислила их квадраты, сумму всех чисел и сумму квадратов чётных ($4^2 + 10^2 + 6^2 = 152$), а также

среднее арифметическое значений, превышающих введенный порог 50 000: $(72\,000 + 91\,000) / 2 = 81\,500$. Объект без свойства value в расчет не попал.



```
anastasia@MacBook-Pro lab1 % node cli.js
=== Обработка массива чисел ===
Введите числа через пробел: 4 7 10 3 6
Введённый массив:          [ 4, 7, 10, 3, 6 ]
Только чётные числа:       [ 4, 10, 6 ]
Квадраты чисел:            [ 16, 49, 100, 9, 36 ]
Сумма чисел:                30
Сумма квадратов чётных:     152
Утроение через applyToEach: [ 12, 21, 30, 9, 18 ]

=== Обработка массива объектов ===
Формат: имя:значение (элемент без значения – объект
без свойства value). Например: Иванов:45000 Кузнецов
Введите объекты: Иванов:45000 Петров:72000 Кузнецов Смирнова:91000
Введённые объекты:
[
  { name: 'Иванов', value: 45000 },
  { name: 'Петров', value: 72000 },
  { name: 'Кузнецов' },
  { name: 'Смирнова', value: 91000 }
]
Объекты со свойством value:
[
  { name: 'Иванов', value: 45000 },
  { name: 'Петров', value: 72000 },
  { name: 'Смирнова', value: 91000 }
]
Введите порог: 50000
Среднее значений > 50000:      81500
```

Рисунок 6 – Работа программы в интерактивном режиме

6 ССЫЛКА НА РЕПОЗИТОРИЙ

Исходный код лабораторной работы размещён в системе контроля версий Git и доступен по ссылке: <https://github.com/nastenka777/Functional-programming>

Код первой лабораторной работы находится в каталоге lab1.

7 ВЫВОДЫ

В ходе выполнения лабораторной работы освоены основные принципы функционального программирования в языке JavaScript. Разработан набор чистых функций для работы с массивами: отбор чётных чисел, возведение элементов в квадрат, фильтрация массива объектов по наличию заданного свойства и суммирование элементов. Реализована функция высшего порядка `applyToEach`, применяющая переданную функцию к каждому элементу массива.

На основе разработанных функций методом композиции получены требуемые математические операции – сумма квадратов чётных чисел и среднее арифметическое значений, превышающих заданный порог. Все функции являются чистыми: они не изменяют переданные аргументы, не обращаются к внешнему изменяемому состоянию и при одинаковых входных данных всегда возвращают одинаковый результат, что подтверждено экспериментально. При реализации применены стрелочные функции, модули ES, оператор расширения и другие современные возможности языка.

Дополнительно реализован интерактивный режим, в котором исходные данные вводит пользователь. Разбор введённых строк также выполняют чистые функции, а ввод-вывод изолирован в отдельном файле, поэтому логика обработки данных не изменилась.

Практическая ценность подхода заключается в том, что чистые функции легко тестируются по отдельности и безопасно объединяются в более сложные вычисления, что повышает читаемость и надёжность программного кода.